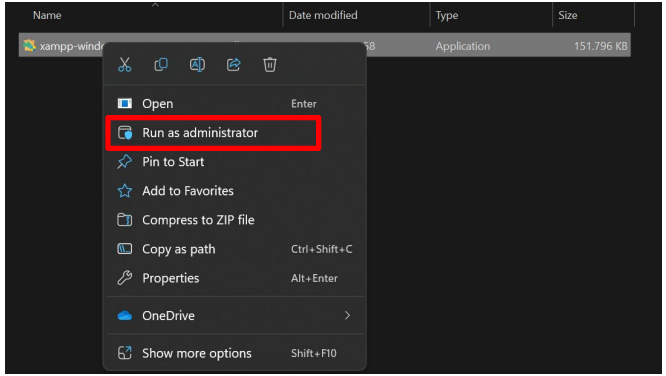
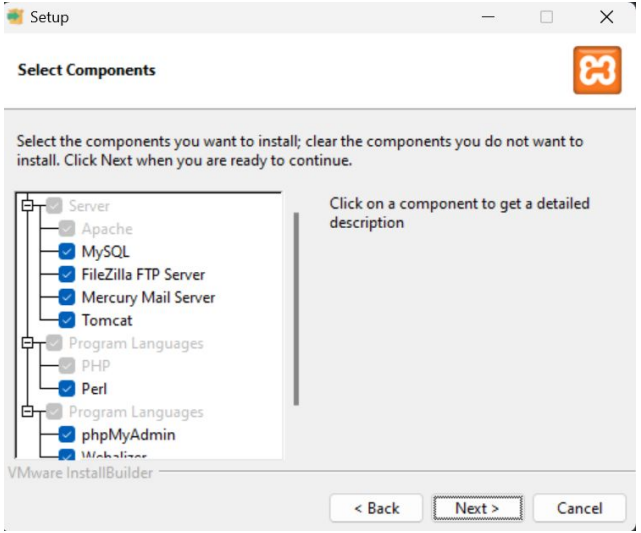
Instalasi XAMPP

1. Unduh *software* XAMPP dan sesuaikan sistem operasi yang dipakai, unduh melalui link: https://www.apachefriends.org/download.html

2. Jalankan *file installer* XAMPP dengan *run as administrator*

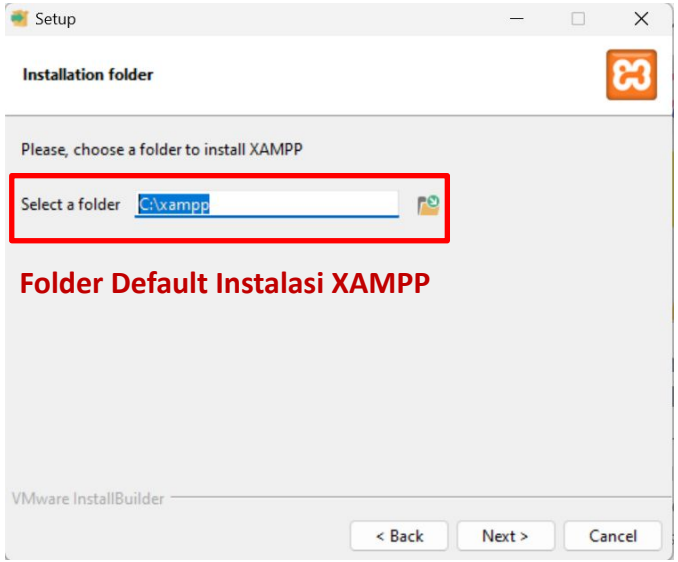


3. Kemudian akan muncul menu seperti tampilan di bawah untuk memilih komponen yang ingin di-*install,* setelahnya klik tombol *next*

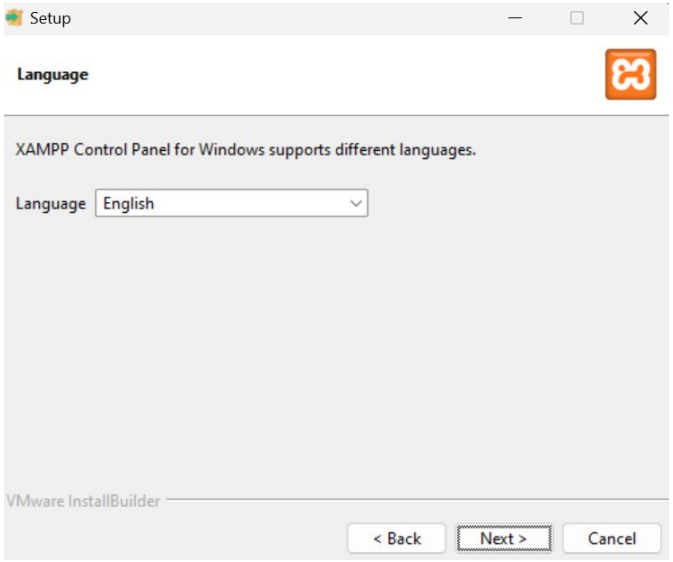


Instalasi XAMPP

4 Lalu, pilihlah lokasi *folder* tempat XAMPP akan di-install atau klik *next* jika ingin melakukan instalasi di *folder default*

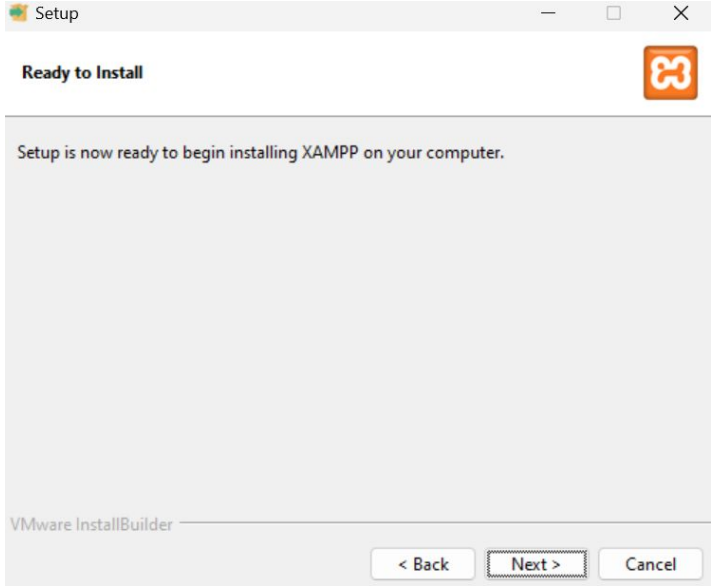


5 Selanjutnya, pilih bahasa yang ingin digunakan di panel kontrol XAMPP



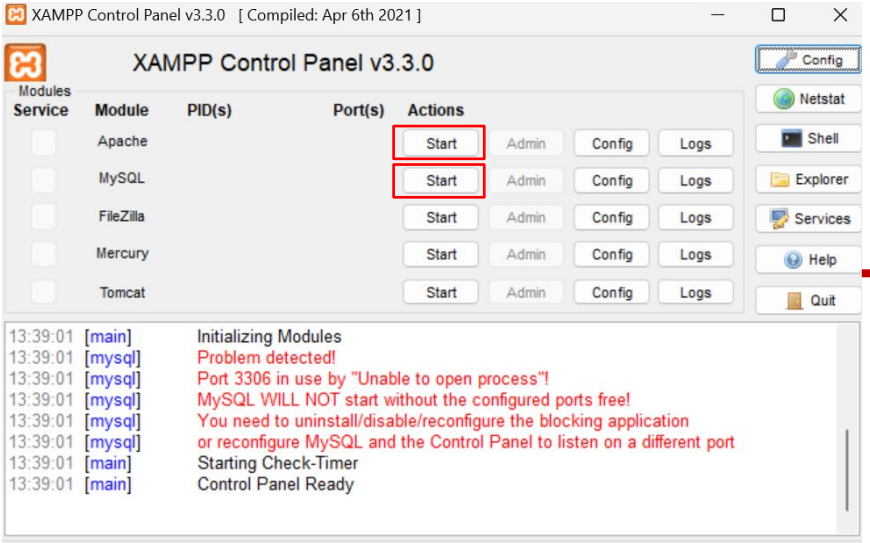
Instalasi XAMPP

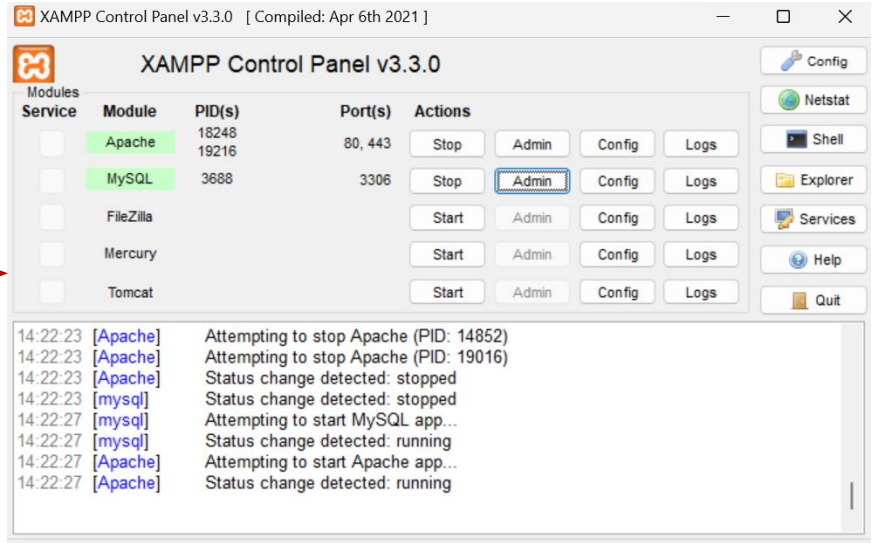
6 Sekarang, *setup* siap untuk melakukan instalasi XAMPP, klik *next* untuk memulai instalasi XAMPP



Tahapan Membuat MySQL Database di XAMPP

1 Buka XAMPP dan aktifkan apache dan MySQL di XAMPP

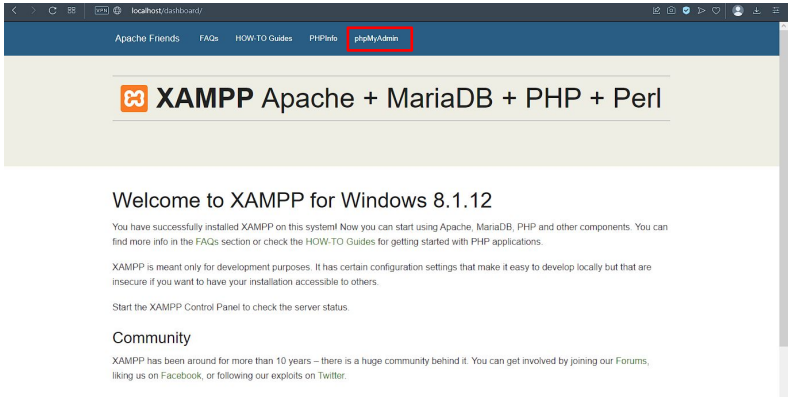




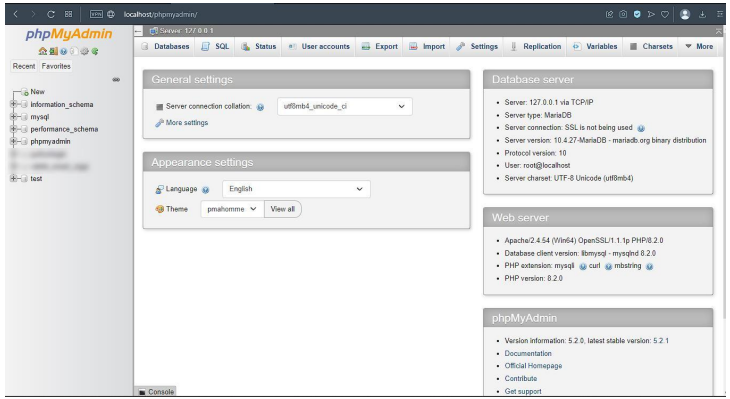
Tahapan Membuat MySQL Database di XAMPP

2

- Buka localhost atau phpMyAdmin pada browser dengan mengakses localhost/dashboard

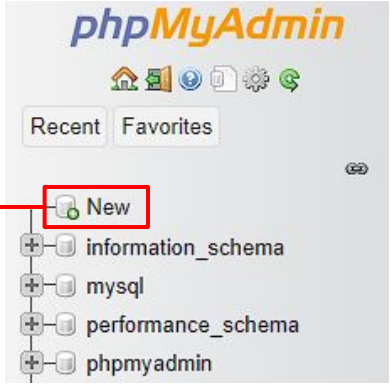


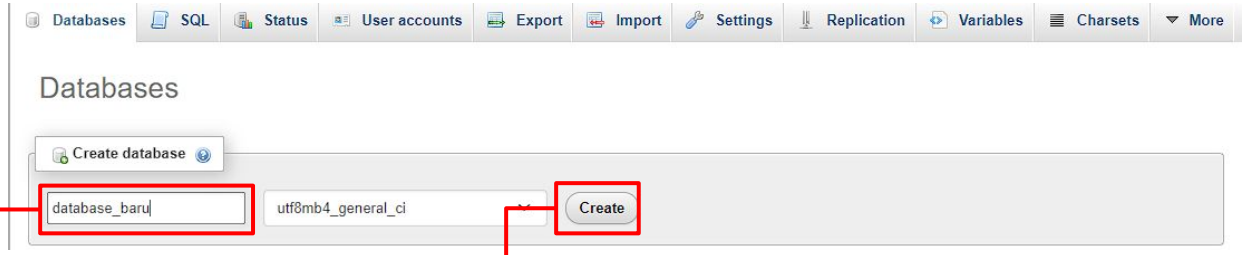
- Atau akses localhost/phpMyAdmin pada browser



Tahapan Membuat MySQL Database di XAMPP

3 Buat *database* MySQL di phpMyAdmin





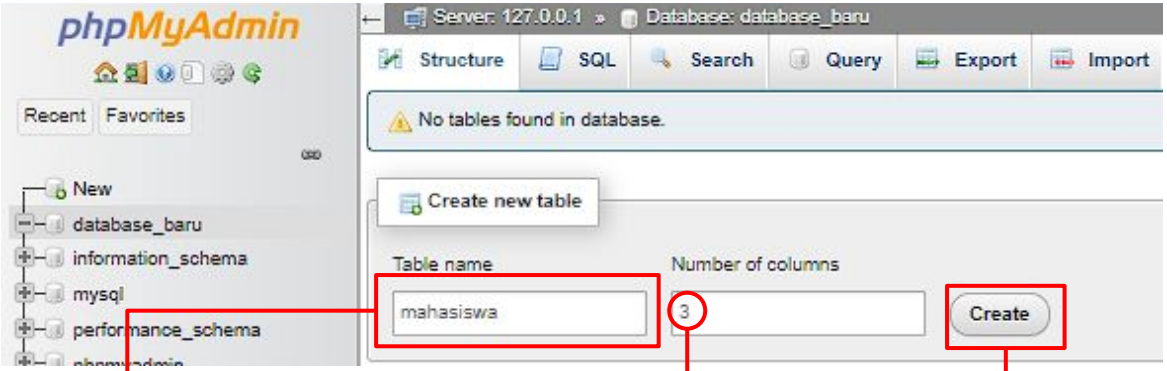
3.1 Klik New

3.2 Isi Nama Database

3.3 Klik Create

Tahapan Membuat MySQL Database di XAMPP

5 Buat Tabel Baru di Database MySQL



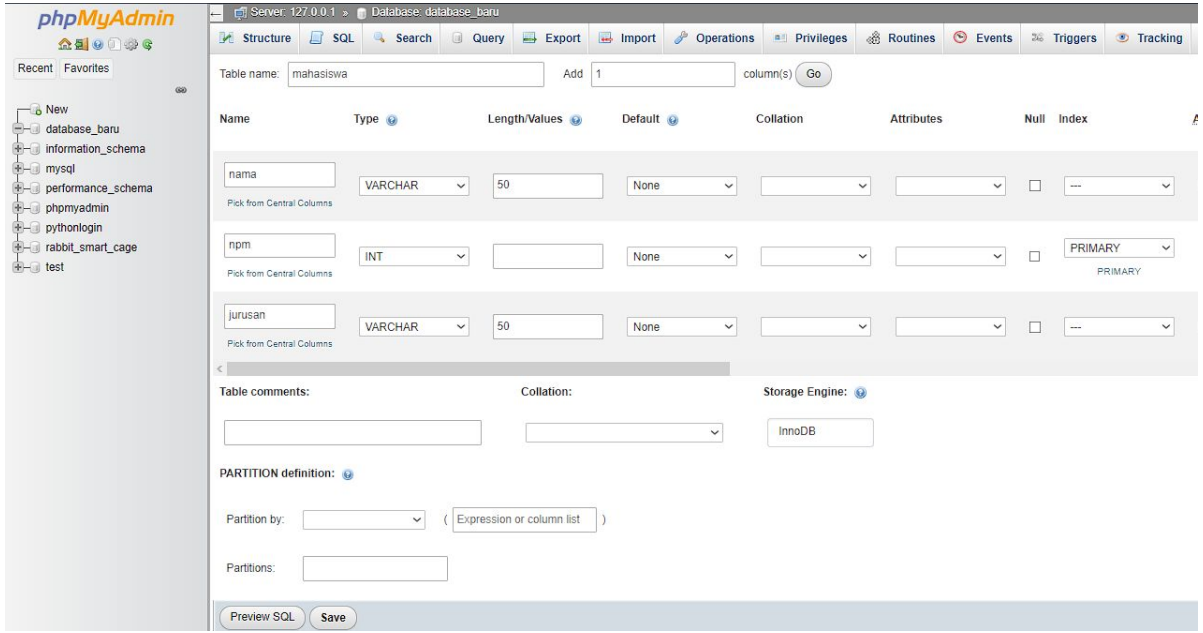
4.1 Isi Nama Tabel dengan "mahasiswa"

4.2 Isi Jumlah Kolom dengan "3"

4.3 Klik Create

Tahapan Membuat MySQL Database di XAMPP

6 Masukkan nama dan tipe data pada kolom, terdapat tiga kolom yaitu nama, NPM, jurusan



1. Kolom “nama” :
   - Name : nama
   - Type : VARCHAR
   - Length : 50

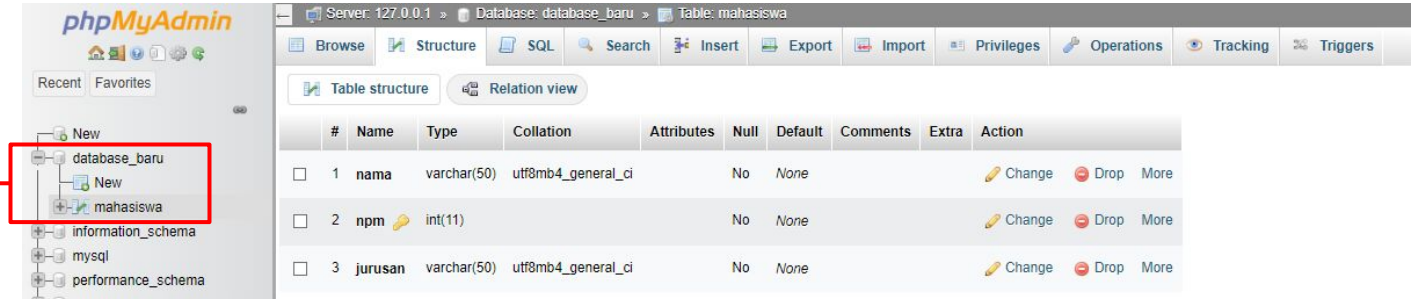
2. Kolom “npm” :
   - Name : npm
   - Type : INT
   - Index : primary

3. Kolom “jurusan” :
   - Name : jurusan
   - Type : VARCHAR
   - Length : 50

Kemudian klik “save”

Tahapan Membuat MySQL Database di XAMPP

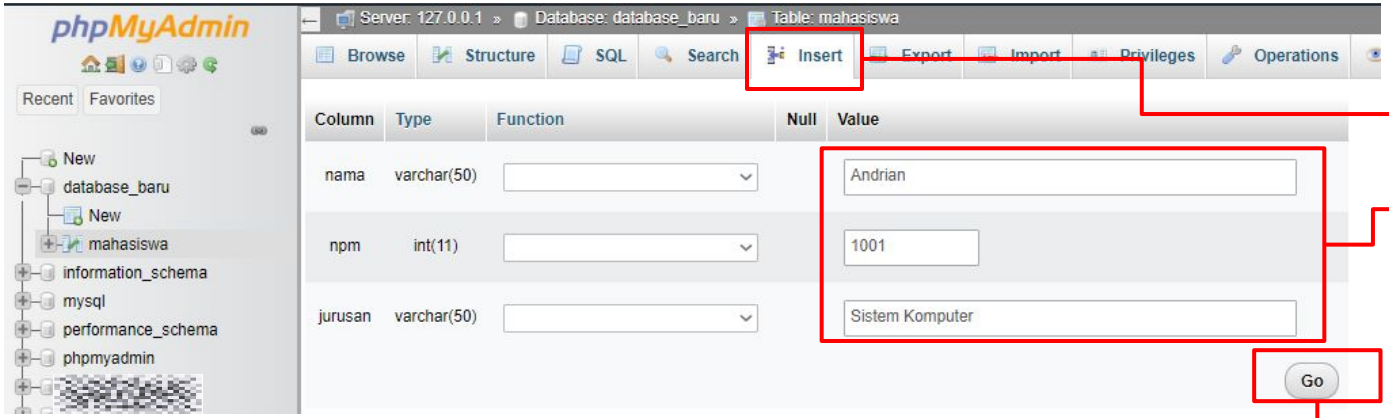
7 Sekarang, database dengan nama "database_baru" dengan tabel "mahasiswa" siap dipakai dan dihubungkan dengan *website*.



7.1 Buka database "database_baru", kemudian pilih tabel "mahasiswa"

Tahapan Membuat MySQL Database di XAMPP

8 Setelah membuat tabel baru, selanjutnya waktunya melakukan input data pada tabel “mahasiswa” pada bagian “insert”

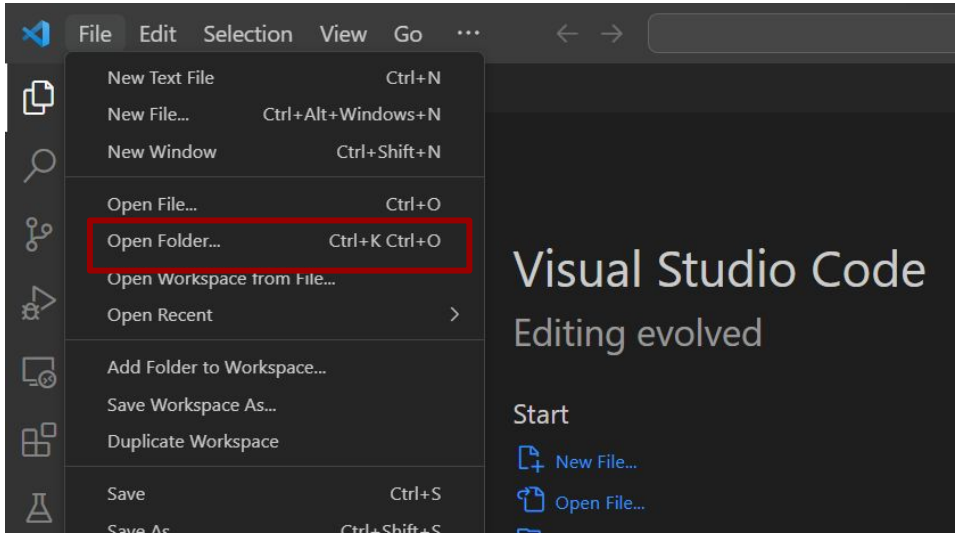


8.1 Buka Tab Insert

8.2 Isi Nilai Kolom

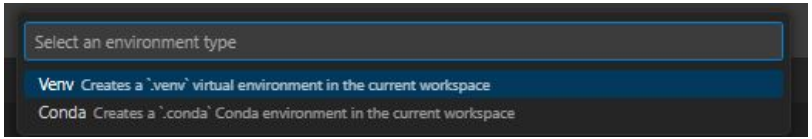
8.3 Klik Go

Membuat Folder “Pertemuan 4”



Buatlah folder pertemuan 4 pada direktori perangkat Anda, kemudian bukalah folder tersebut di visual studio code.

Kemudian buatlah virtual environment baru dengan shortcut ctrl + shift + p



Install Flask

Buka terminal vscode dengan shortcut ctrl + shift + `

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS D:\DGX\Kursus Semester 1\Pertemuan 3\Contoh Kasus> & "d:/DGX/Kursus Semester 1/Pertemuan 3/Contoh Kasus/.venv/Scripts/Activate.ps1"
(.venv) PS D:\DGX\Kursus Semester 1\Pertemuan 3\Contoh Kasus>
```

Kemudian ketikkan python -m pip install flask

Install Flask-MySQLdb & Buat file app.py

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS D:\DGX\Kursus Semester 1\Pertemuan 3\Contoh Kasus> & "d:/DGX/Kursus Semester 1/Pertemuan 3/Contoh Kasus/.venv/Scripts/Activate.ps1"
(.venv) PS D:\DGX\Kursus Semester 1\Pertemuan 3\Contoh Kasus>
```

Ketikkan python -m pip install Flask-MySQLdb

Kemudian buatlah file baru dengan nama app.py

Penjelasan masing-masing baris kode dapat dilihat di *slide* selanjutnya.

Menghubungkan Flask ke MySQL Database pada file app.py

Kode Program app.py

```
from flask import Flask, render_template, url_for, redirect, request, session, flash
from flask_mysqldb import MySQL

app = Flask(__name__)
app.secret_key="pertemuan4"

#Konfigurasi Koneksi Database MySQL
app.config['MYSQL_HOST'] = 'localhost'
app.config['MYSQL_USER'] = 'root'
app.config['MYSQL_PASSWORD'] = ''
app.config['MYSQL_DB'] = 'database_baru'
mysql = MySQL(app)


if __name__ == '__main__' :
    app.run(debug=True, port='3000')
```

Menghubungkan Flask ke MySQL Database pada file app.py

Import Modul

```
1  from flask import Flask, render_template, url_for, redirect, request, session, flash
2  from flask_mysqldb import MySQL
```

- Flask: Digunakan untuk membuat dan mengelola aplikasi web dengan Flask.
- render_template: Digunakan untuk merender template HTML.
- url_for: Untuk membuat URL untuk fungsi yang ditentukan.
- redirect: Digunakan untuk mengarahkan pengguna ke URL lain.

Menghubungkan Flask ke MySQL Database pada file app.py

Import Modul

```
from flask import Flask, render_template, url_for, redirect, request, session, flash
from flask_mysqldb import MySQL
```

- request: Untuk mengelola permintaan HTTP.
- session: Untuk menyimpan informasi di antara permintaan.
- flash: Untuk menampilkan pesan flash, seperti pemberitahuan atau pesan sukses setelah operasi tertentu.
- Flask_mysqldb: Adalah ekstensi Flask untuk menghubungkan Flask dengan database MySQL.

Menghubungkan Flask ke MySQL Database pada file app.py

Inisialisasi Aplikasi Flask dan Pengaturan Kunci Rahasia

```
app = Flask(__name__)
app.secret_key="pertemuan4"
```

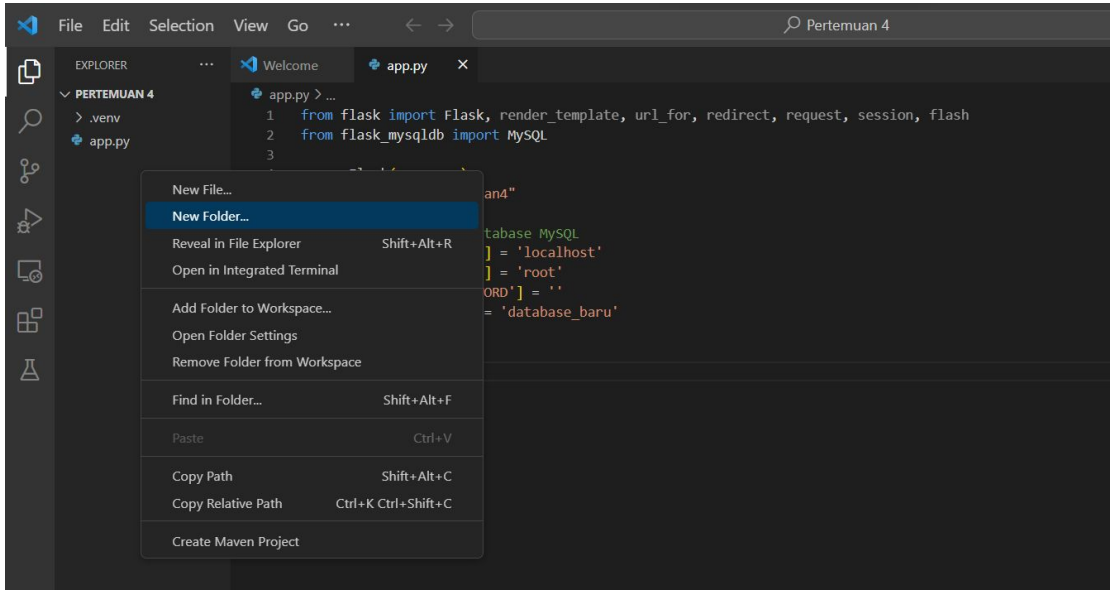
- Membuat instance aplikasi Flask.
- Menetapkan kunci rahasia untuk sesi Flask (digunakan untuk menyimpan data sesi yang aman).

Konfigurasi Koneksi Database MySQL

```
#Konfigurasi Koneksi Database MySQL
app.config['MYSQL_HOST'] = 'localhost'
app.config['MYSQL_USER'] = 'root'
app.config['MYSQL_PASSWORD'] = ''
app.config['MYSQL_DB'] = 'database_baru'
mysql = MySQL(app)
```

- Menetapkan konfigurasi koneksi ke database MySQL, seperti alamat host, nama pengguna, kata sandi, dan nama database.
- Membuat instance dari kelas MySQL dan menghubungkannya ke aplikasi Flask.

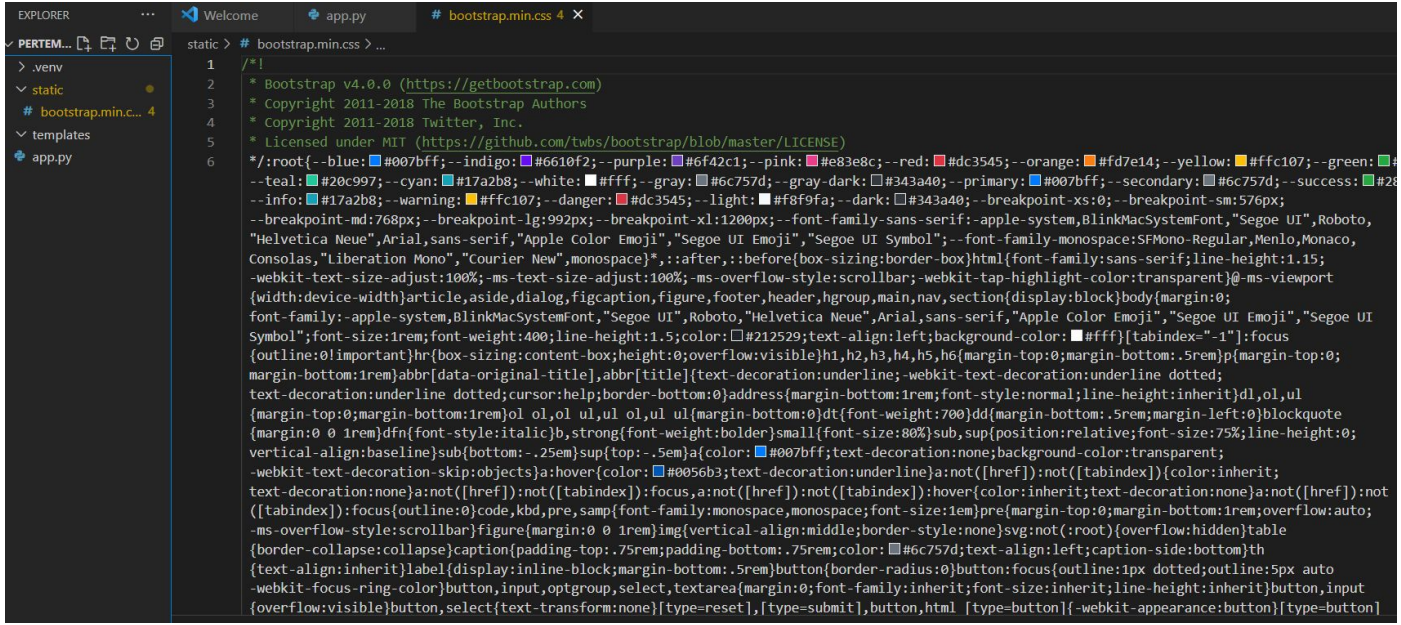
Buatlah Folder “templates” dan “static”



Buatlah folder dengan nama “templates” dan “static” dengan cara klik kanan pada bagian explorer seperti gambar di samping.

Kemudian pilih opsi new folder.

Buatlah Folder “templates” dan “static”



Lalu letakkan file bootstrap.min.css pada folder static.

Buatlah file login.html dan tempatkan pada folder templates

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Tutorial Bootstrap Flask</title>
    <link rel="stylesheet" type=text/css href="{{ url_for('static', filename='bootstrap.min.css') }}">
</head>
<body>
    <div class="container">
        <div class="row">
            <div class="col-lg-12">
                <form action="{{ url_for('login') }}" method="post">
                    <div class="mb-3 mt-3">
                        <label for="email" class="form-label">Email : </label>
                        <input type="email" class="form-control" placeholder="Input Email" name="Email">
                    </div>

                    <div class="mb-3">
                        <label for="password" class="form-label">Password : </label>
                        <input type="password" class="form-control" placeholder="Input password" name="password">
                    </div>

                    <button type="submit" class="btn btn-primary">Submit</button>
                </form>
            </div>
        </div>
    </div>
</body>
</html>
```

Buatlah file login.html dan tempatkan pada folder templates

```
<link rel="stylesheet" type=text/css href="{{ url_for('static', filename='bootstrap.min.css') }}">
```

Menghubungkan file CSS Bootstrap (bootstrap.min.css) yang disimpan di direktori statis menggunakan fungsi Flask url_for.

Buatlah file login.html dan tempatkan pada folder templates

```
<form action="{{ url_for('login') }}" method="post">
```

- action="{{ url_for('login') }}": Menentukan URL atau rute Flask yang akan menangani data formulir ketika formulir tersebut di-submit. Dalam hal ini, formulir akan mengirim data ke rute yang disebut 'login'. url_for adalah fungsi Flask yang digunakan untuk menghasilkan URL untuk rute tertentu berdasarkan nama rute.

- method="post": Menentukan metode HTTP yang akan digunakan oleh formulir untuk mengirimkan data. Dalam hal ini, metode POST digunakan. Metode POST digunakan untuk mengirim data formulir ke server dengan cara yang lebih aman dan dapat menyembunyikan informasi sensitif, seperti kata sandi, dari URL.

Tambahkan rute utama “/” pada file app.py

```
@app.route('/', methods=['GET','POST'])
def login() :
    if request.method == 'POST' :
        return redirect(url_for('mahasiswa'))
    return render_template('login.html')
```

Deklarasi Rute

- Menggunakan dekorator @app.route('/') untuk mendeklarasikan bahwa fungsi login menangani rute utama ('/').
- Menambahkan parameter methods=['GET', 'POST'] untuk menunjukkan bahwa fungsi ini dapat menanggapi kedua metode HTTP GET dan POST.

```
if request.method == 'POST' :
        return redirect(url_for('mahasiswa'))
    return render_template('login.html')
```

- Memeriksa apakah metode permintaan (HTTP method) adalah POST. Jika iya, itu berarti formulir login telah di-submit.
- Pada kondisi ini, fungsi ini mengarahkan pengguna ke rute 'mahasiswa' menggunakan redirect(url_for('mahasiswa')).

Tambahkan rute “/mahasiswa” pada file app.py

```python
@app.route('/mahasiswa')
def mahasiswa() :
    cursor = mysql.connection.cursor()
    cursor.execute(''' SELECT * FROM mahasiswa ''')
    mahasiswa = cursor.fetchall()
    cursor.close()

    return render_template('mahasiswa.html', mahasiswa=mahasiswa)
```

- Membuat objek kursor menggunakan mysql.connection.cursor(). Kursor digunakan untuk menjalankan perintah SQL pada database.
- Mengeksekusi query SQL untuk mengambil semua data dari tabel 'mahasiswa'.
- Mengambil semua baris hasil query dan menyimpannya dalam variabel 'mahasiswa'. Hasilnya biasanya berupa tuple atau list.
- Menutup kursor setelah selesai menggunakannya. Ini adalah praktik yang baik untuk memastikan bahwa sumber daya database dibersihkan setelah digunakan.

Mengembalikan hasil render template 'mahasiswa.html' dengan data mahasiswa yang telah diambil dari database. Data mahasiswa disediakan ke template menggunakan parameter mahasiswa.

Buat file “mahasiswa.html”

Buat file “mahasiswa.html” dengan menyalin file “login.html” dan melakukan penamaan ulang menjadi file “mahasiswa.html”.

Kemudian hapus kode di dalam elemen

```
<div class="col-lg-12">
    <form action="{{ url_for('login') }}" method="post">
        <div class="mb-3 mt-3">
            <label for="email" class="form-label">Email : </label>
            <input type="email" class="form-control" placeholder="Input Email" name="Email">
        </div>

        <div class="mb-3">
            <label for="password" class="form-label">Password : </label>
            <input type="password" class="form-control" placeholder="Input password" name="password">
        </div>

        <button type="submit" class="btn btn-primary">Submit</button>
    </form>
</div>
```

Tambahkan kode berikut :

```html
<div class="col-lg-12">
    <h3 class="one">Welcome Mahasiswa</h3>

    <table class="table">
        <thead>
            <tr>
                <th scope="col">Nama</th>
                <th scope="col">NPM</th>
                <th scope="col">Jurusan</th>
                <th scope="col">Aksi</th>
            </tr>
        </thead>

        <tbody>
            {% for m in mahasiswa %}
            <tr>
                <td>{{ m[0] }}</td>
                <td>{{ m[1] }}</td>
                <td>{{ m[2] }}</td>

                <td>
                    <a href="/mahasiswa/edit/{{ m[1] }}" class="btn btn-warning" role="button">Edit</a>
                    <a href="/mahasiswa/delete/{{ m[1] }}" class="btn btn-danger" role="button"
                        onclick="return confirm('Apakah Anda Yakin Ingin Menghapus Data Ini?')">
                        Delete</a>
                </td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
</div>
```

Tambahkan kode berikut :

<tbody>:
Ini adalah elemen HTML yang digunakan untuk menandai bagian tubuh dari sebuah tabel. Biasanya digunakan untuk mengelompokkan baris-baris data.

{% for m in mahasiswa %}:
Ini adalah sintaksis kontrol aliran pada Jinja. Ini memulai loop for yang akan mengulang setiap elemen dalam variabel mahasiswa.

<tr>...</tr>:
Ini adalah elemen HTML yang digunakan untuk menandai baris dalam tabel.

{{ m[0] }}, {{ m[1] }}, {{ m[2] }}:
Ini adalah sintaksis Jinja yang digunakan untuk menyisipkan nilai dari setiap elemen dalam variabel m ke dalam HTML. m[0], m[1], dan m[2] mewakili elemen-elemen pada setiap baris data mahasiswa.

Tambahkan kode berikut :

<a href="/mahasiswa/edit/{{ m[1] }}" ... >Edit</a>:
Ini adalah tautan yang mengarah ke rute '/mahasiswa/edit/' dengan parameter ID mahasiswa (m[1]). Ini dapat digunakan untuk mengarahkan pengguna ke halaman edit mahasiswa.

<a href="/mahasiswa/delete/{{ m[1] }}" ... >Delete</a>:
Ini adalah tautan yang mengarah ke rute '/mahasiswa/delete/' dengan parameter ID mahasiswa (m[1]). Terdapat juga atribut onclick yang menampilkan konfirmasi pengguna sebelum menghapus data.

{% endfor %}:
Ini mengakhiri blok loop for.

Tambahkan kode berikut :

Tampilan halaman mahasiswa akan seperti di bawah ini :

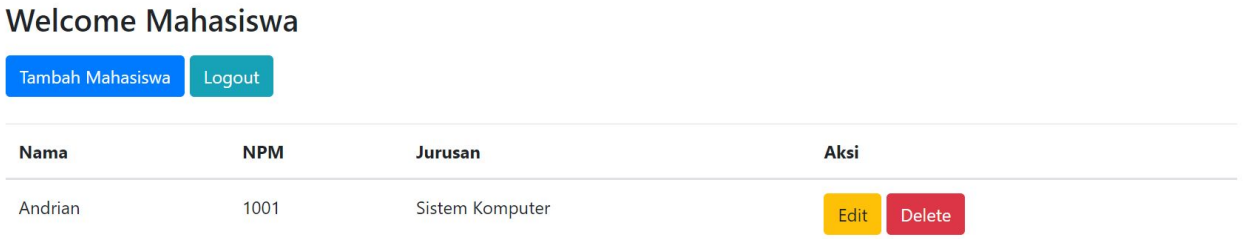
Welcome Mahasiswa

Nama	NPM	Jurusan	Aksi
Andrian	1001	Sistem Komputer	Edit Delete

Tambahkan tombol “Tambah Mahasiswa” & “Logout”

Tambahkan kode “mt-4” pada class di elemen h3 “Welcome Mahasiswa” kemudian tambahkan tombol “Tambah Mahasiswa” & “Logout” di bawah tulisan “Welcome Mahasiswa” dengan kode berikut :

```
1  <h3 class="one mt-4">Welcome Mahasiswa</h3>
2
3  <a href="/mahasiswa/tambah" class="btn btn-primary mt-2 mb-4" role="button">Tambah Mahasiswa</a>
4  <a href="" class="btn btn-info mt-2 mb-4" role="button">Logout</a>
```



Modifikasi app.py – Fungsi Logout

Pada app.py, tambahkan route (‘/logout’) agar fungsi logout dapat berfungsi

```
@app.route('/logout')
def logout() :
    session.pop('logged_in',None)
    return redirect(url_for('login'))
```

Kemudian tambahkan kode program {{ url_for('logout') }} pada file mahasiswa.html di tombol logout pada bagian href seperti di bawah ini :

```
<a href="{{ url_for('logout') }}" class="btn btn-info mt-2 mb-4" role="button">Logout</a>
```

Atribut href menentukan URL atau rute yang akan diakses ketika tautan diklik. Dalam hal ini, url_for('logout') digunakan untuk menghasilkan URL yang sesuai dengan rute Flask yang disebut 'logout'.

Menambah Route “Tambah Mahasiswa”

Pada file app.py, tambahkan route (‘/mahasiswa/tambah’) dengan method GET dan POST.

- Method GET digunakan untuk memanggil form input pengguna yang berisi field nama mahasiswa, npm dan jurusan.
- Method POST digunakan untuk manangani input user saat tombol submit dipilih. Data yang disubmit akan masuk ke database database_baru tabel mahasiswa.

Untuk mengambil input user dari form kita gunakan “request.form”. Selanjutnya pengguna akan diarahkan ke halaman mahasiswa dengan data yang sudah ditambahkan. Alur route (‘/mahasiswa/tambah’) akan menjadi seperti ini :

```
if method == 'GET’:
        #panggil form input
else:
        #baca input user dan simpan ke MySQL
```

Menambah Route “Tambah Mahasiswa”

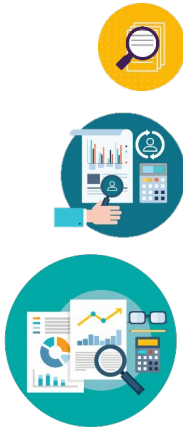
Kode Program route(‘/mahasiswa/tambah’)

```
@app.route('/mahasiswa/tambah', methods=['GET','POST'])
def tambahmahasiswa():
    if request.method == 'GET'  :
        return render_template('mahasiswa/add.html')
    else :
        nama = request.form['nama']
        npm = request.form['npm']
        jurusan = request.form['jurusan']

        cursor = mysql.connection.cursor()
        cursor.execute(''' INSERT INTO mahasiswa(nama,npm,jurusan) VALUES(%s,%s,%s) '''
                       ,(nama,npm,jurusan))
        mysql.connection.commit()
        cursor.close()
        return redirect(url_for('mahasiswa'))
```

Membuat Form Tambah Mahasiswa

Selanjutnya buat folder “mahasiswa” di dalam folder “templates” dan buat file add.html, kode programnya adalah seperti berikut :

```html
{% extends "layout.html" %}

{% block content %}
<form action="/mahasiswa/tambah" method="post">
    <div class="mb-13 mt-3">
        <label for="nama" class="form-label">Nama :</label>
        <input type="input" class="form-control" placeholder="Isi Nama" name="nama" required autofocus>
    </div>

    <div class="mb-3">
        <label for="npm" class="form-label">NPM :</label>
        <input type="input" class="form-control" placeholder="Isi NPM" name="npm" required>
    </div>

    <div class="mb-3">
        <label for="jurusan" class="form-label">Jurusan :</label>
        <input type="input" class="form-control" placeholder="Isi Jurusan" name="jurusan" required >
    </div>

    <button type="submit" class="btn btn-primary">Submit</button>

</form>
{% endblock %}
```



```
{% extends "layout.html" %}
{% block title %}Tambah Mahasiswa{% endblock %}

{% block content %}
<form action="/mahasiswa/tambah" method="post">
    <div class="mb-13 mt-3">
        <label for="nama" class="form-label">Nama
:</label>
        <input type="input" class="form-control"
placeholder="Isi Nama" name="nama" required
autofocus>
    </div>

    <div class="mb-3">
        <label for="npm" class="form-label">NPM
:</label>
        <input type="input" class="form-control"
placeholder="Isi NPM" name="npm" required>
    </div>

    <div class="mb-3">
        <label for="jurusan"
class="form-label">Jurusan :</label>
        <input type="input" class="form-control"
placeholder="Isi Jurusan" name="jurusan" required
>
    </div>

    <button type="submit" class="btn
btn-primary">Submit</button>

</form>
{% endblock %}
```

{% extends "layout.html" %}

Element “extends” merupakan element yang merujuk kepada *base template* yaitu layout.html. Sementara kode program di samping merupakan *child template.*

Base dan child template merupakan bagian dari template inheritance (pewarisan) di Flask. Template inheritance berfungsi untuk membuat kerangka layout tanpa harus mengulang-mengulang hal yang sama.

Template inheritance memungkinkan hanya ada 1 template dasar/kerangka yang dapat diganti oleh child template yang lebih spesifik.

Template Inheritance pada Flask

Kode Program login.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible"
content="IE=edge">
    <meta name="viewport"
content="width=device-width, initial-scale=1.0">
    <title>Tutorial Bootstrap Flask</title>
    <link rel="stylesheet" type=text/css
href="{{ url_for('static',
filename='bootstrap.min.css') }}">
</head>
```

Kode Program mahasiswa.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible"
content="IE=edge">
    <meta name="viewport"
content="width=device-width, initial-scale=1.0">
    <title>Tutorial Bootstrap Flask</title>
    <link rel="stylesheet" type=text/css
href="{{ url_for('static',
filename='bootstrap.min.css') }}">
</head>
```

Coba perhatikan potongan kode dari login.html dan mhs.html. Terlihat persamaan yang sebenarnya tidak perlu diulang. Hal ini bisa dibuat lebih efisien dengan menerapkan template inheritance dengan membuat 1 kerangka yang menyimpan *tag* yang sama yang disebut base *template.*

Template Inheritance pada Flask – Base Template

Buat file layout.html di folder templates yang merupakan base template. File ini menyimpan tag yang digunakan oleh seluruh halaman seperti tag html, body dan sebagainya. Untuk elemen yang berbeda pada setiap halaman, gunakan block yang nantinya akan di-override oleh child template.

Di dalam kelas row isi dari halaman diganti menjadi block content yang isinya bisa bermacam-macam saat dioverride child template,

```
{% block content %}{% endblock %}
```

Template Inheritance pada Flask – Base Template

Kode Program layout.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Tutorial Bootstrap Flask</title>
    <link rel="stylesheet" type=text/css href="{{ url_for('static', filename='bootstrap.min.css')
}}">
</head>
<body>
    <div class="container">
        <div class="row">
            <div class="col-lg-12">
                {% block content %}{% endblock %}
            </div>
        </div>
    </div>
</body>
</html>
```

Template Inheritance pada Flask – Child Template

Kode Program add.html

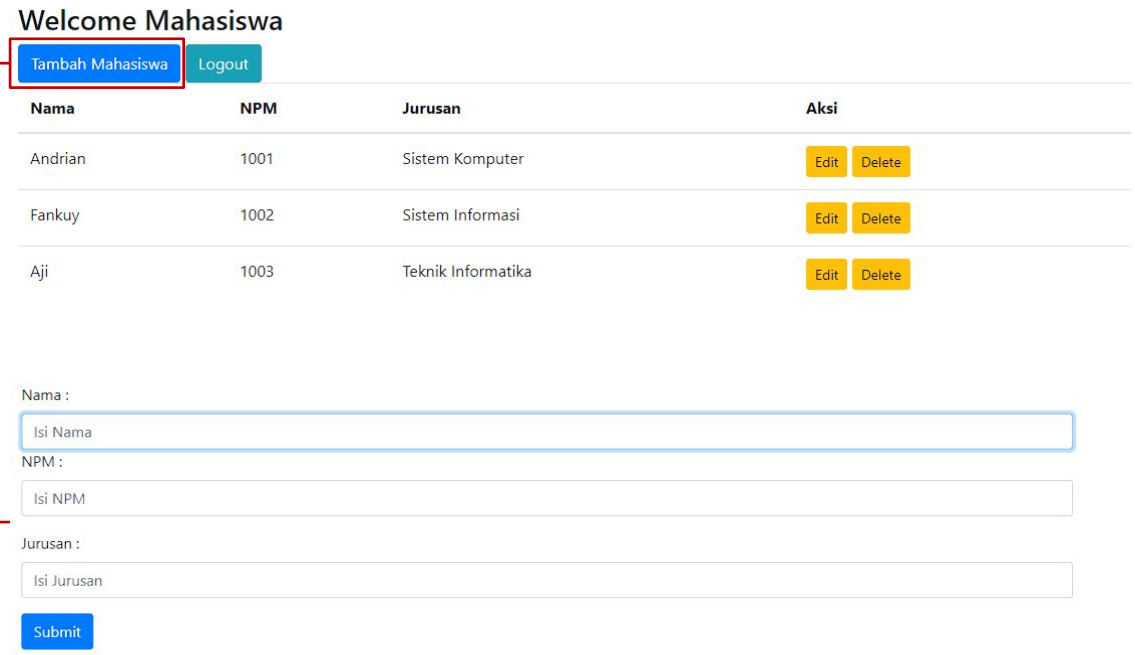
```
{% extends "layout.html" %}

{% block content %}
<form action="/mahasiswa/tambah" method="post">
    <div class="mb-13 mt-3">
        <label for="nama" class="form-label">Nama :</label>
        <input type="input" class="form-control" placeholder="Isi Nama" name="nama" required autofocus>
    </div>

    <div class="mb-3">
        <label for="npm" class="form-label">NPM :</label>
        <input type="input" class="form-control" placeholder="Isi NPM" name="npm" required>
    </div>

    <div class="mb-3">
        <label for="jurusan" class="form-label">Jurusan :</label>
        <input type="input" class="form-control" placeholder="Isi Jurusan" name="jurusan" required >
    </div>

    <button type="submit" class="btn btn-primary">Submit</button>

</form>
{% endblock %}
```

Demo Input Mahasiswa

1. Klik tombol tambah mahasiswa

2. Isikan nilai untuk nama, NPM, dan Jurusan

3. Tekan tombol submit



Membuat Route “Edit Mahasiswa”

Pada file app.py, tambahkan route (‘/mahasiswa/edit/<int:npm>’) dengan method GET dan POST. <int:npm> merupakan routing dinamis untuk mengambil npm. Method GET digunakan untuk memanggil form edit pengguna yang sudah bernilai dari field nama dan jurusan seperti pada route(‘/mahasiswa/tambah’).

Sedangkan method POST digunakan untuk menangani update/edit terbaru saat tombol submit dipilih. Data yang di-submit akan terupdate di database “database_baru” tabel “mahasiswa”.

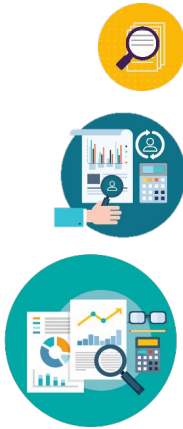
```
If method == ‘GET’ :
        #baca data dari database berdasarkan npm
else :
        #baca input user terbaru dan simpan ke database MySQL
```

Membuat Route “Edit Mahasiswa”

Untuk mengambil data mahasiswa dengan npm tertentu menggunakan klausa WHERE dan ambil dengan fetchone() karena data yang diambil hanya 1 record data. Sedangkan untuk mengupdate data mahasiswa gunakan perintah UPDATE dengan terlebih dulu membaca data mahasiswa yang sudah di-update di form edit

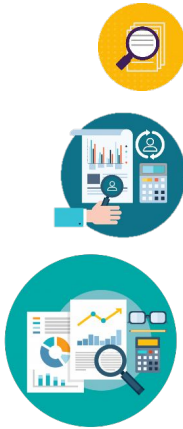
```
UPDATE nama_table
SET field1 = nilai1, field2 = nilai2, ...
WHERE kondisi;
```

Kode program route(‘/mahasiswa/edit/<int:npm>’) ada di slide selanjutnya



Membuat Route “Edit Mahasiswa”

```
@app.route('/mahasiswa/edit/<int:npm>', methods=['GET','POST'])
def editmahasiswa(npm) :
    if request.method == 'GET' :
        cursor = mysql.connection.cursor()
        cursor.execute('''
        SELECT *
        FROM mahasiswa
        WHERE npm = %s
        ''', (npm, ))
        mahasiswa = cursor.fetchone()
        cursor.close()

        return render_template('mahasiswa/edit.html', mahasiswa=mahasiswa)

    else :
        nama = request.form['nama']
        jurusan = request.form['jurusan']

        cursor = mysql.connection.cursor()
        cursor.execute('''
        UPDATE mahasiswa
        SET
            nama = %s,
            jurusan = %s
        WHERE
            npm = %s
        ''', (nama,jurusan,npm))

        mysql.connection.commit()
        cursor.close()
        return redirect(url_for('mahasiswa'))
```



Membuat Form “Edit Mahasiswa”

Selanjutnya pada folder templates/mahasiswa buatlah file edit.html, file tersebut akan digunakan untuk menampilkan halaman edit data mahasiswa.

```
{% extends "layout.html" %}

{% block content %}
<form action="/mahasiswa/edit/{{ mahasiswa[1] }}" method=post>
    <div class="mb-13 mt-3">
        <label for="nama" class="form-label">Nama : </label>
        <input type="input" class="form-control" placeholder="Ubah
Nama" name="nama" value="{{ mahasiswa[0] }}">
    </div>

    <div class="mb-3">
        <label for="jurusan" class="form-label">Jurusan : </label>
        <input type="input" class="form-control" placeholder="Ubah
Jurusan" name="jurusan" value="{{ mahasiswa[2] }}">
    </div>

    <button type="submit" class="btn btn-primary">Update</button>
</form>
{% endblock %}
```

Karena sebelumnya sudah dibuat template inheritance maka pada kode program ini dapat langsung memanggil block layout dengan perintah :

{% extends "layout.html" %}

Membuat Form “Edit Mahasiswa”

```
{% extends "layout.html" %}

{% block content %}
<form action="/mahasiswa/edit/{{ mahasiswa[1] }}" method=post>
    <div class="mb-13 mt-3">
        <label for="nama" class="form-label">Nama : </label>
        <input type="input" class="form-control" placeholder="Ubah
Nama" name="nama" value="{{ mahasiswa[0] }}">
    </div>

    <div class="mb-3">
        <label for="jurusan" class="form-label">Jurusan : </label>
        <input type="input" class="form-control" placeholder="Ubah
Jurusan" name="jurusan" value="{{ mahasiswa[2] }}">
    </div>

    <button type="submit" class="btn btn-primary">Update</button>
</form>
{% endblock %}
```

Terakhir tuliskan block content untuk menampilkan form yang berisi nilai dari atribut nama dan jurusan. Untuk mendapatkan nilai dari setiap atribut tabel, gunakan array dengan :

1. Indeks 0 -> nama
2. Indeks 1 -> npm
3. Indeks 2 -> jurusan

Demo Update Data Mahasiswa

Klik tombol edit untuk mengedit data mahasiswa

Demo Update Data Mahasiswa

Nama :

Andrian

Jurusan :

Sistem Komputer

Update

Kemudian tampilan halaman akan berubah seperti di atas, lalu pengguna dapat mengubah nilai dari nama dan jurusan kemudian klik tombol ”*update*” untuk memperbaharui data, sehingga data akan berubah seperti pada tampilan di bawah :

Welcome Mahasiswa

Tambah Mahasiswa | Logout

Nama	NPM	Jurusan	Aksi
Andrian	1001	Ilmu Komputer	Edit Delete
Fankuy	1002	Sistem Informasi	Edit Delete
Aji	1003	Teknik Informatika	Edit Delete

Membuat Route Delete Mahasiswa

Buat route(‘mahasiswa/delete/<int:npm>’) pada app.py dengan method GET, untuk melakukan perintah penghapusan data dapat mengguakana klausa DELETE dengan perintah sebagai berikut :

DELETE

FROM nama_table

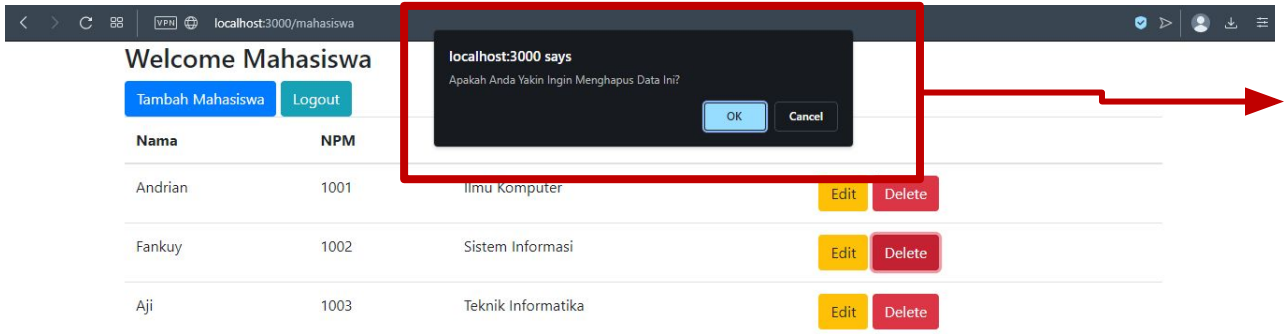
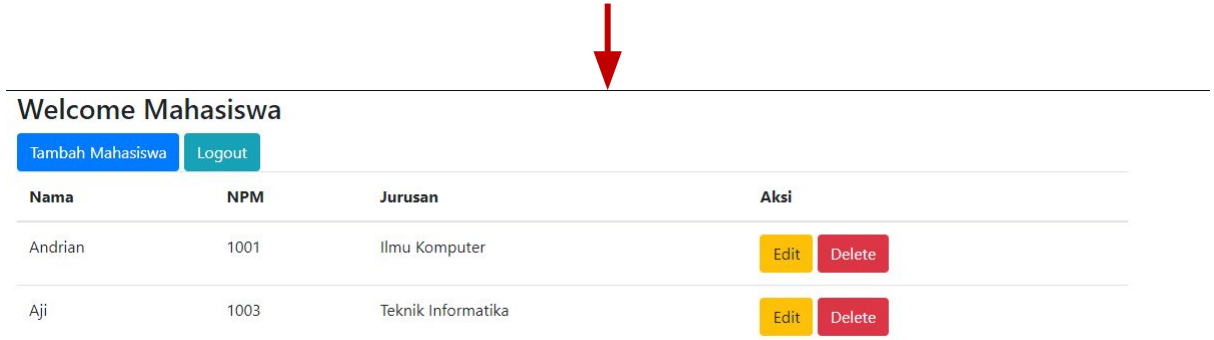
WHERE kondisi

Kode program route (‘mahasiswa/delete/<int:npm>’) ada dl slide berikutnya

Membuat Route Delete Mahasiswa

```
@app.route('/mahasiswa/delete/<int:npm>', methods=['GET'])
def deletemahasiswa(npm) :
    if request.method == 'GET' :
        cursor = mysql.connection.cursor()
        cursor.execute('''
        DELETE
        FROM mahasiswa
        WHERE npm=%s
        ''', (npm, ))
        mysql.connection.commit()
        cursor.close()

        return redirect(url_for('mahasiswa'))
```

Demo Delete Data Mahasiswa



Klik tombol OK